

Penetration Testing Lab Report

Web Application Employee Portal IDOR Vulnerability Exploitation

Project Title: Broken Access Control Investigation & Flag Extraction

Target Application: StaffHub (localhost:8080)

Author : Sudip Bastola

Assessment Date: August 11, 2026

Assessment Type: Controlled Academic / Lab Web Application Security Assessment

1. Vulnerability Information

Vulnerability Name:	Insecure Direct Object Reference (IDOR) / Broken Access Control
CWE ID:	CWE-639: Authorization Bypass Through User-Controlled Key
OWASP Category:	A01:2021 – Broken Access Control
Severity:	Medium
Location:	http://localhost:8080/profile.php?id=1
Vulnerable Parameter:	id

2. Description

An Insecure Direct Object Reference (IDOR) occurs when a web application uses user-supplied input to directly access database records or internal system objects without performing adequate server-side authorization checks. Because the application trusts the client's request parameters blindly, an attacker can manipulate these identifiers to bypass intended access controls.

During the authorized assessment of the StaffHub web application, individual employee profiles were accessed through a URL structure utilizing the id GET parameter (e.g., <http://localhost:8080/profile.php?id=1>). By manually altering the integer value assigned to this parameter, the application rendered different employee records across the organization without validating if the requester had the authorization to view them. This represents a fundamental access-control failure because identity verification and data access boundaries are delegated to the user's browser rather than enforced by the backend server.

Unauthorized access to employee records poses significant privacy and operational risks. Exposing internal directory details allows malicious actors to conduct reconnaissance, gather personal information for social engineering, or harvest operational intelligence. Furthermore, reaching high-profile or high-privileged accounts—such as the CEO's profile—amplifies the impact by exposing sensitive executive data, internal credentials, or organizational secrets that could compromise the wider enterprise.

3. Tools Used

- **Brave Browser**
Description: Used to access the StaffHub web application, navigate the employee directory, inspect the profile URL, manually modify the id parameter, and observe the employee profile returned by the application.

4. Step-by-Step Exploitation

Step 1 - Accessing the Employee Directory

Action: Navigated to the main employee directory within the StaffHub web application and clicked the "View" option corresponding to the first listed employee entry.

Reason: This initial step was executed to map the navigation flow and observe how the application handles requests for individual user profiles.

Observation : The browser successfully loaded and rendered the selected employee profile page.

Security Significance: Baseline behavior was established, demonstrating that employee profiles are retrieved dynamically via user interactions within the interface.

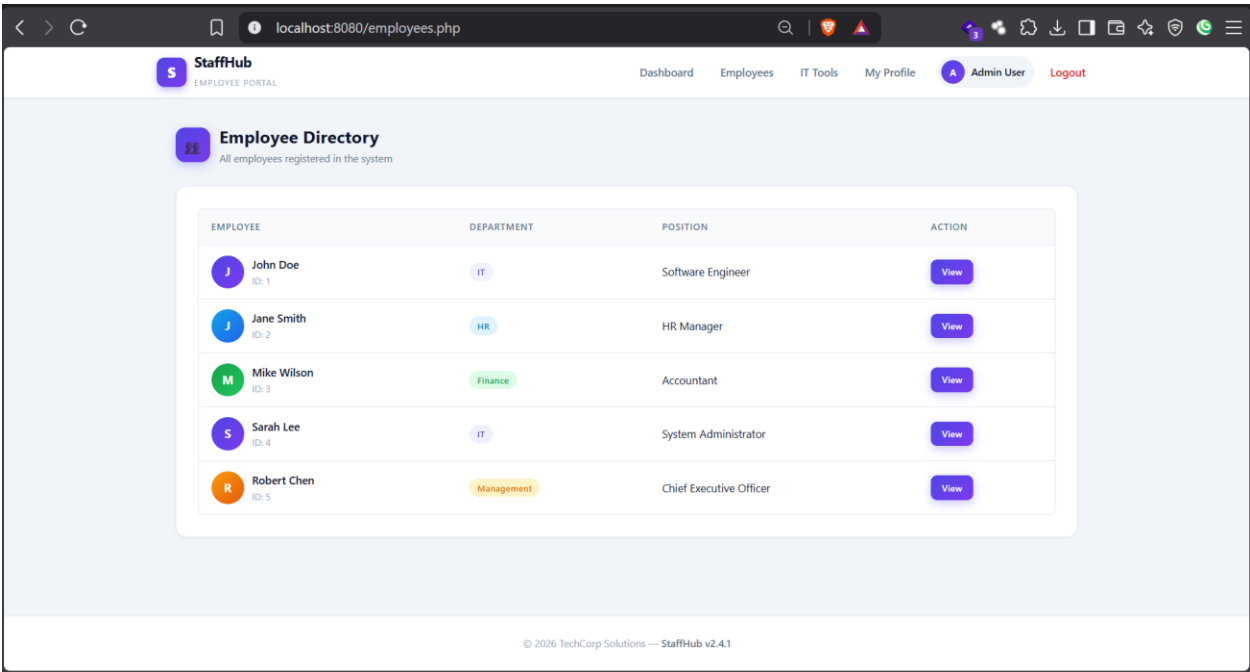


Figure 1 StaffHub Employee Directory

Step 2 Identifying the Profile URL and Object Reference

Action: Inspected the web browser's address bar after loading the first employee profile.

Reason: To identify how the backend application references and retrieves specific user profile objects.

URL / Parameter: `http://localhost:8080/profile.php?id=1`

Parameter: `id`

Observation : The application loaded the URL containing `profile.php?id=1`, where `id=1` served as a direct reference to the database record of the requested employee.

Security Significance: Identifying user-controlled object identifiers (such as sequential integer IDs in GET parameters) highlights key attack surfaces for potential access control bypasses during an assessment.

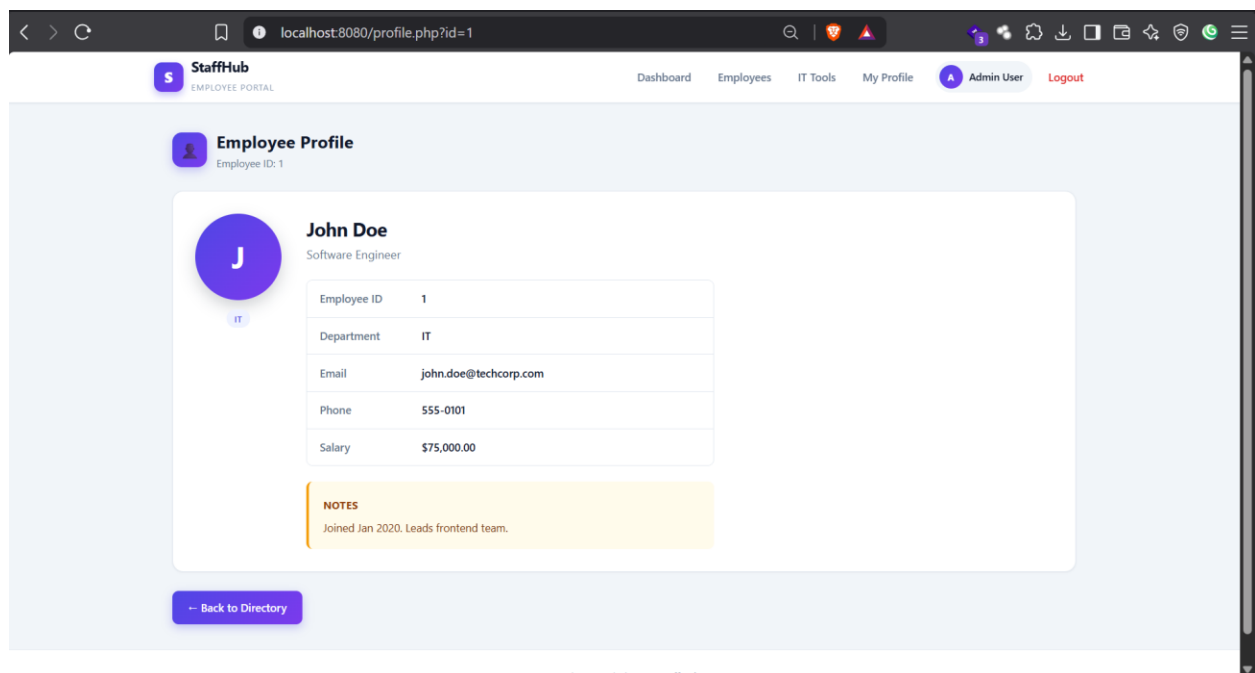


Figure 2 Employee Profile URL Showing the User Controlled `id=1` Parameter

Step 3 - Manually Modifying the ID Parameter

Action: Interpreted and manually modified the GET parameter value in the browser address bar from id=1 to alternate numeric identifiers (e.g., id=2, id=3).

Reason: To test whether the application enforces server-side authorization checks when a user explicitly requests object identifiers belonging to other accounts.

URL / Parameter: `http://localhost:8080/profile.php?id=[MODIFIED_VALUE]`

Parameter: id

Observation : The application accepted the manually modified parameter values and loaded the corresponding employee records.

Security Significance: Demonstrates that the application does not restrict requests based on session context, permitting arbitrary lookup of sequential user records.

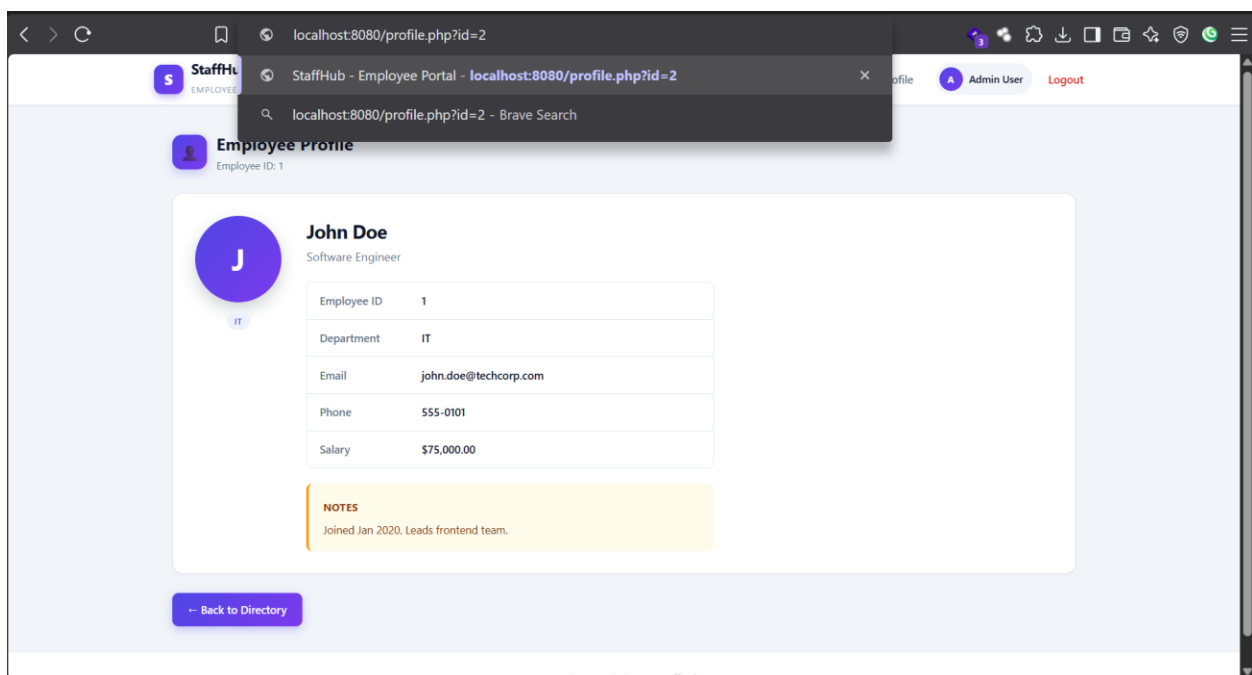


Figure 3 Manual Modification of the id Parameter Resulting in Access to Another Employee profile

Step 4 -Confirming Unauthorized Object Access

Action: Reviewed the retrieved profile information returned after sending the modified id values.

Reason: To verify whether the application actively enforced object-level permission controls before rendering data.

URL / **Parameter:** `http://localhost:8080/profile.php?id=[MODIFIED_VALUE]`
Parameter: id

Observation : The server returned full profile data for different employees without requesting re-authentication or displaying an access-denied error.

Security Significance: This confirmed an active Insecure Direct Object Reference (IDOR) vulnerability. The application relies entirely on client-controlled input to fetch database records without checking server-side authorization rules.

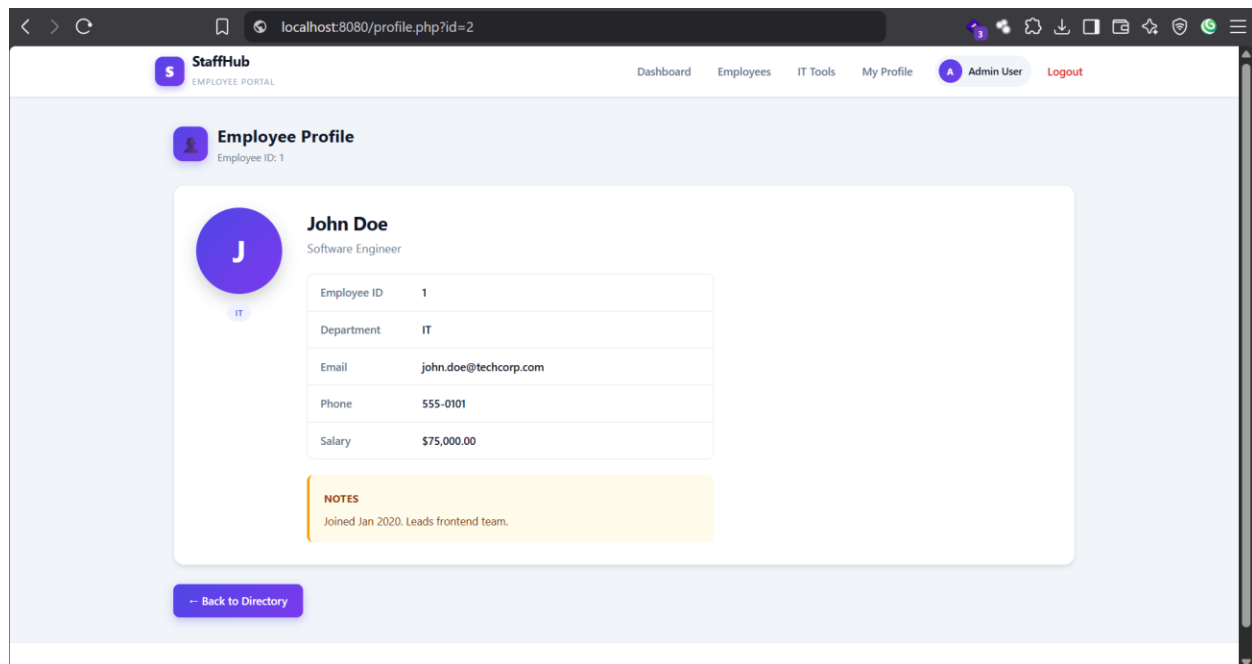


Figure 4 IDOR Testing Showing Access to a Different Employee Profile by Modifying the Object Identifier

Step 5 - Accessing Employee ID 5

Action: Incremented the parameter value specifically to <http://localhost:8080/profile.php?id=5>.

Reason: To evaluate the scope of the vulnerability and determine if higher-privileged or executive accounts were similarly exposed through object parameter manipulation.

URL / Parameter: <http://localhost:8080/profile.php?id=5>

Parameter: id

Observation : The page rendered successfully, revealing the account profile corresponding to the organization's Chief Executive Officer (CEO).

Security Significance: Proves that the access control flaw is systemic across the endpoint, allowing lower-privileged users to inspect sensitive executive-level records.

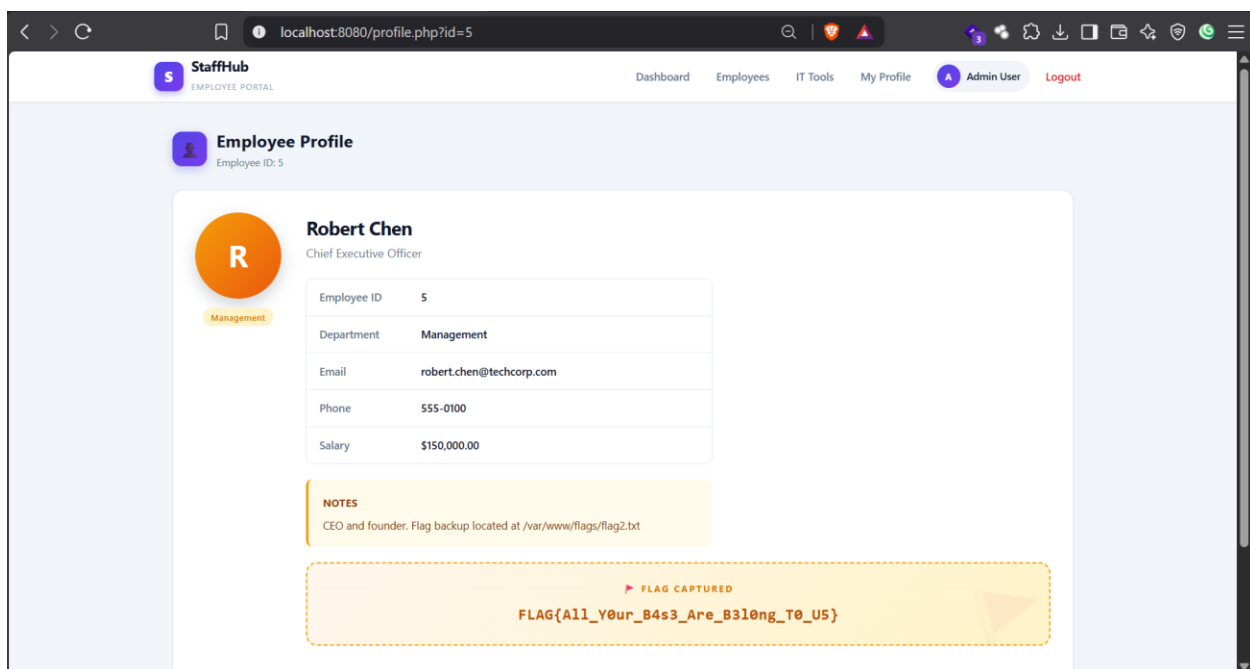


Figure 5 Access to the CEO Profile Through the id=5 Parameter

Step 6 - Flag Capture

Action: Examined the rendered content on the CEO's profile page (id=5).

Reason: To capture the lab's proof-of-concept flag, validating successful exploitation.

URL / Parameter: http://localhost:8080/profile.php?id=5

Parameter: id

Observation : The challenge flag was exposed on the profile page and retrieved successfully.

Security Significance: The flag serves as definitive proof that improper access control permits unauthorized retrieval of restricted target data.

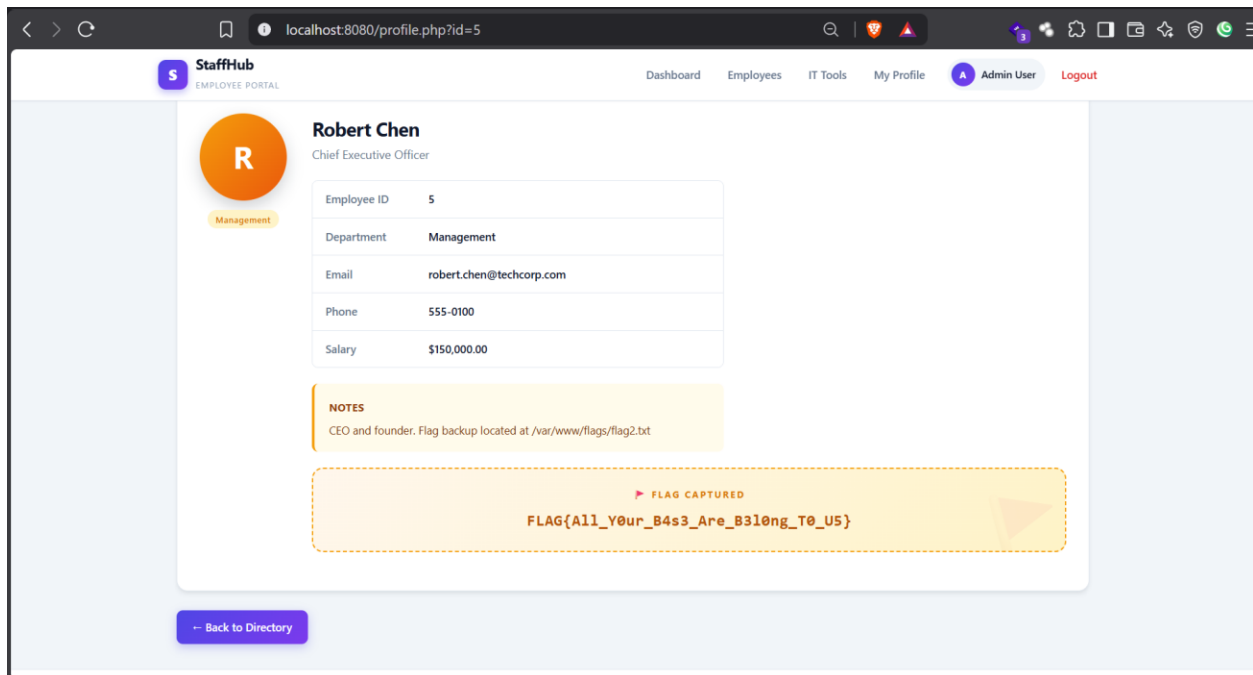


Figure 6 Flag Displayed on the CEO Profile After Successful IDOR Exploitaion

5. Proof of Exploitation

The evidence below confirms that modifying the user-controlled id parameter directly granted access to unauthorized records, culminating in access to the CEO account and target flag.

Exact Flag: FLAG{All_Y0ur_B4s3_Are_B3l0ng_T0_U5}

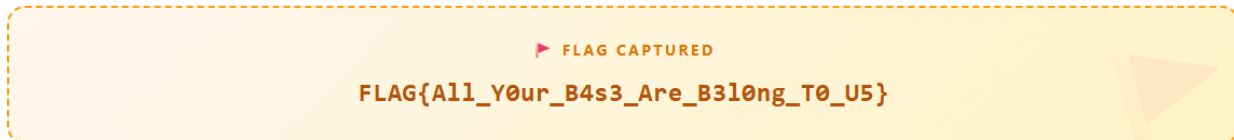


Figure 7 Captured Flag

6. Business Impact

Demonstrated in the Lab

During testing on the StaffHub application, changing the id parameter directly exposed internal employee profile data, including the CEO profile and the challenge flag, without performing any authorization checks.

Potential Real-World Impact

In an enterprise environment, an IDOR vulnerability of this nature presents significant business risks. An attacker could automate the enumeration of sequential profile IDs to harvest an entire corporate directory, exposing personally identifiable information (PII), work emails, phone numbers, and operational roles. Exposing executive profiles elevates the threat, as sensitive communications, internal notes, or proprietary disclosures hosted on high-level accounts can lead to corporate espionage, spear-phishing campaigns, and reputational damage.

7. Remediation Recommendation

To resolve this Insecure Direct Object Reference vulnerability, StaffHub should implement robust, server-side access control mechanisms:

- **Server-Side Authorization Checks:** Every incoming request to profile.php must validate the user's session server-side before querying or returning profile data. The application must never trust client-supplied parameters alone.
- **Object-Level Access Control (ABAC/RBAC):** Prior to serving record data, the application logic must verify whether the authenticated user has explicit permission to view the requested object.
- **Adopt Deny-by-Default Architecture:** Enforce a baseline security policy where all access requests to internal objects are rejected unless explicit authorization is established for the active session.
- **Avoid Relying on Client-Controlled Identifiers:** Numeric database keys (id=1) should not act as standalone access tokens. If direct references are necessary, pair them with robust session checks or replace sequential integer IDs with cryptographically secure Indirect Object.
- **Centralized Authorization Logic:** Authorization checks should be implemented consistently across all API endpoints and web routes that handle employee records or internal objects.
- **Routine Access Control Testing:** Integrate automated and manual security testing focused on parameter tampering across all application endpoints.

References

Common Weakness Enumeration, n.d. *CWE-639:Authorization Bypass Through User-Controlled Key*. [Online]

Available at: <https://cwe.mitre.org/data/definitions/639.html>
[Accessed 11 08 2026].

OWASP, n.d. *A01:2021 – Broken Access Control*. [Online]

Available at: https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/index.html

[Accessed 11 08 2026].

OWASP, n.d. *Testing for Insecure Direct Object References*. [Online]

Available at: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References

[Accessed 11 08 2026].

PortSwigger, n.d. *IDOR*. [Online]

Available at: <https://portswigger.net/web-security/access-control/idor>

[Accessed 11 08 2026].